

# SESIÓN 7 Separación de Responsabilidades

UI vs Lógica · Arquitectura modular · 1 hora

Fase 2 — Sistema base · Marzo 2025 · Sesión 7 de 18

|             |                                           |
|-------------|-------------------------------------------|
| ■ Duración  | 1 hora                                    |
| ■ Fase      | Marzo — Sistema base                      |
| ■ Sesión    | 7 de 18                                   |
| ■ Resultado | Estructura de carpetas clara + JS modular |

## ■ 1 Idea central de la sesión

**Un programa bien hecho no es uno que funciona. Es uno en el que cada pieza hace exactamente su trabajo y nada más.**

En esta sesión aprenderemos a separar el código en responsabilidades claras: la UI se ocupa de mostrar, la lógica se ocupa de pensar, el estado se ocupa de recordar. Esta separación es la base de toda ingeniería de software profesional.

## ■ 2 Conexión con lo que ya sabes

| Concepto previo                                          | Cómo se conecta con hoy                                                                    |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Scratch: bloques separados (Movimiento, Sonido, Eventos) | En Scratch ya separabas qué hacía cada bloque. Hoy hacemos lo mismo, pero con archivos.    |
| HTML / CSS separados                                     | Ya viste que HTML es estructura y CSS es apariencia. Ahora añadimos: JavaScript es lógica. |
| Estado global (Sesión 5)                                 | El estado vive en estado.js. La UI en ui.js. La lógica del juego en su propio archivo.     |
| Navegación (Sesión 6)                                    | navegar() es una función de app.js. No está mezclada con el tablero del Tic Tac Toe.       |

## ■ 3 Separación de responsabilidades

### ¿Qué significa que algo tenga una sola responsabilidad?

Imagina que en un restaurante el cocinero también cobra, limpia y hace publicidad. Cuando algo falla, no sabes quién tiene el problema. Lo mismo pasa en el código.

| Archivo    | Responsabilidad                            | No debe hacer               |
|------------|--------------------------------------------|-----------------------------|
| index.html | Estructura de la página (qué divs existen) | Lógica ni estilos complejos |
| styles.css | Apariencia visual (colores, tamaños)       | Decisiones de negocio       |

|                             |                                                  |                                |
|-----------------------------|--------------------------------------------------|--------------------------------|
| <code>estado.js</code>      | Guardar el estado global de la app               | Renderizar HTML                |
| <code>app.js</code>         | Orquestrar la app: navegación, inicio            | Reglas del juego               |
| <code>ui.js</code>          | Renderizar el estado en pantalla                 | Cambiar el estado directamente |
| <code>tic-tac-toe.js</code> | Lógica del Tic Tac Toe: turnos, victoria, empate | Manipular el DOM               |

## ⇒ 4 Diseño antes de código

### La estructura de carpetas del proyecto

```
mini-juegos-app/  
■  
■■■ index.html ← única página HTML  
■■■ styles.css ← todos los estilos  
■  
■■■ js/  
■ ■■■ estado.js ← estado global (sesión 5)  
■ ■■■ app.js ← orquestador: inicia la app, maneja navegación  
■ ■■■ ui.js ← renderiza el estado en pantalla  
■ ■  
■ ■■■ juegos/  
■ ■■■ tic-tac-toe.js ← lógica pura del juego  
■ ■■■ ahorcado.js ← lógica pura del juego (sesión 12)  
■  
■■■ assets/  
■■■ (imágenes, sonidos en el futuro)
```

#### ■ **Pregunta de diseño:** ¿Por qué ui.js está separado de tic-tac-toe.js?

Porque el juego no debería saber cómo se dibuja en pantalla. Si mañana cambiamos el diseño visual, no tocamos la lógica del juego. **Eso es robustez.**

## ■ 5 Ejemplo aplicado al proyecto

### Comparemos dos versiones del mismo código:

#### ■ Versión mezclada (como no debe quedar)

```
// En app.js – hace demasiadas cosas a la vez

function alHacerClic(fila, col) {
  estado.tablero[fila][col] = estado.turno // lógica de juego
  document.getElementById('celda-'+fila+'-'+col) // UI
    .textContent = estado.turno // UI
  if (comprobarVictoria()) { // lógica de juego
    alert('¡Ganaste!') // UI
    estado.pantallaActual = 'resultado' // navegación
  }
}
```

### ■ Versión separada (como debe quedar)

```
// En tic-tac-toe.js – solo lógica
function procesarMovimiento(fila, col) {
  estado.tablero[fila][col] = estado.turno
  if (comprobarVictoria()) {
    navegar('resultado') // delega la navegación
  } else {
    cambiarTurno()
    actualizarUI() // delega el renderizado
  }
}

// En ui.js – solo renderizado
function renderizarTablero() {
  estado.tablero.forEach((fila, i) => {
    fila.forEach((valor, j) => {
      document.getElementById(`celda-${i}-${j}`).textContent = valor
    })
  })
}
```

## ■ 6 Uso guiado de IA

### ■ Qué pedirle a la IA

- «Revisa si esta función hace más de una cosa»
- «¿Este código mezcla UI con lógica?»
- «Ayúdame a mover la lógica de victoria a su propio archivo»
- «Explicame qué responsabilidad tiene cada función»

### ■■ Qué revisar siempre

- Que no mezcle lógica + UI en el mismo archivo
- Que las funciones tengan nombres claros
- Que no use `document.getElementById` en `tic-tac-toe.js`
- Que siga la estructura de carpetas del proyecto

■ **Error típico de la IA:** poner todo en un solo archivo enorme y llamarlo «más simple». Más corto no significa mejor. Un archivo de 500 líneas que hace todo es peor que 5 archivos de 100 líneas bien separados.

## ■ 7 Preguntas de pensamiento crítico

|     |                                                                                                          |                                                                                |
|-----|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| P 1 | Si quiero cambiar cómo se ve el tablero pero no las reglas del juego, ¿qué archivo debo tocar? ¿Por qué? | Pista: Separar UI de lógica hace que los cambios sean localizados.             |
| P 2 | Si <code>estado.js</code> importa cosas de <code>ui.js</code> , ¿qué problema puede causar eso?          | Pista: Las dependencias circulares y el acoplamiento.                          |
| P 3 | ¿Cómo sabemos que una función hace demasiadas cosas?                                                     | Pista: Si no podemos describir su propósito en una frase sencilla.             |
| P 4 | ¿Por qué <code>tic-tac-toe.js</code> no debería saber nada de HTML?                                      | Pista: Para que la lógica sea reutilizable e independiente del entorno visual. |

## ■ 8 Mini-reto de la sesión (1 hora)

### 1 Diseña la estructura (10 min)

Dibuja en papel los archivos del proyecto y qué función vive en cada uno. Solo nombres de funciones, sin código.

### 2 Crea la estructura de carpetas (5 min)

Crea los archivos vacíos: `estado.js`, `app.js`, `ui.js`, `juegos/tic-tac-toe.js`.

### 3 Mueve el estado (10 min)

Pasa el objeto `'estado'` y sus funciones de modificación a `estado.js`. Comprueba que `app.js` puede importarlo.

### 4 Mueve la navegación (10 min)

`navegar()` va a `app.js` y `actualizarUI()` va a `ui.js`. ¿Qué necesita importar cada archivo?

### 5 Prepara el módulo del juego (15 min)

En `tic-tac-toe.js` crea funciones vacías: `iniciarJuego()`, `procesarMovimiento()`, `comprobarVictoria()`. Solo nombres y comentarios.

6

Revisión crítica (10 min)

Pregunta a la IA: '¿Esta estructura respeta la separación de responsabilidades?' Lee su respuesta con ojo crítico.

■ **Resultado esperado:** Al terminar la hora, tienes la estructura de carpetas creada y el código distribuido en sus archivos correctos. Lo importante es que **cada archivo sepa cuál es su trabajo**.

■ 9 Qué NO hacer

| Error                                                   | Por qué es un problema                                                                 |
|---------------------------------------------------------|----------------------------------------------------------------------------------------|
| Poner toda la lógica en index.html                      | Las etiquetas <script> en HTML son para cargar archivos, no para escribir toda la app. |
| Usar document.getElementById() dentro de tic-tac-toe.js | La lógica del juego no debe saber nada del DOM. Eso es trabajo de ui.js.               |
| Tener un app.js de 300 líneas que hace todo             | Si un archivo crece sin control, es señal de que necesita dividirse.                   |
| Pedirle a la IA la estructura sin entenderla            | Si la IA te da los archivos y no sabes por qué, no has aprendido nada.                 |
| Cambiar el estado desde ui.js                           | La UI obedece al estado; nunca lo modifica directamente. Regla del sistema.            |

■ Regla de oro de esta sesión

Una función bien diseñada hace una sola cosa y la hace bien.  
Un archivo bien diseñado tiene una sola razón para cambiar.

*Si no puedes explicar en una frase qué hace un archivo, es que hace demasiado.*

➡■ Próxima  
sesión

Sesión 8 — Uso responsable de IA  
Aprenderás a qué pedirle a la IA, dónde se equivoca y cómo ser tú la arquitecta del sistema, no ella.